

МОДУЛЬ 1

АДМИНИСТРИРОВАНИЕ LINUX

СБОРКА И УСТАНОВКА ПРОГРАММ

ТИПЫ ПОСТАВКИ ПО



	Бинарный установочный пакет	Установочный архив	Исходный код
Поставка	Исполняемые файлы и библиотеки, файлы ресурсов	Исполняемые файлы и библиотеки, файлы ресурсов	Исходный код
Инструкции по установке	Интегрированы в установочный пакет и выполняются автоматически при начале установки	Инструкции находятся в отдельном файле (Readme), описывающем все этапы установки	Сборочные инструкции (Makefile, config) Инструкции по установке (Readme)

ИНСТАЛЯТОРЫ

Dedbian



Slackware



RedHat



dpkg



apt-get

Zypper

yum
yellowdog updater modified

МЕНЕДЖЕР DEBIAN PACKAGER

ИНСТАЛЯЦИЯ ПАКЕТОВ

Форма:

`dpkg [keys] package_file`

Ключи:

-i PACKAGE – установить пакет PACKAGE

-r PACKAGE – удалить пакет PACKAGE

-l – получить список установленных пакетов

-L PACKAGE – получить список файлов, установленных с пакетом

PACKAGE

-S FILE – узнать с каким пакетом был установлен файл FILE

ПАКЕТНЫЙ МЕНЕДЖЕР АРТ

Advanced Packaging Tool – набор утилит для управления программными пакетами в операционных системах основанных на Debian. АРТ предоставляет дружественную надстройку над DPKG.

Позволяет:

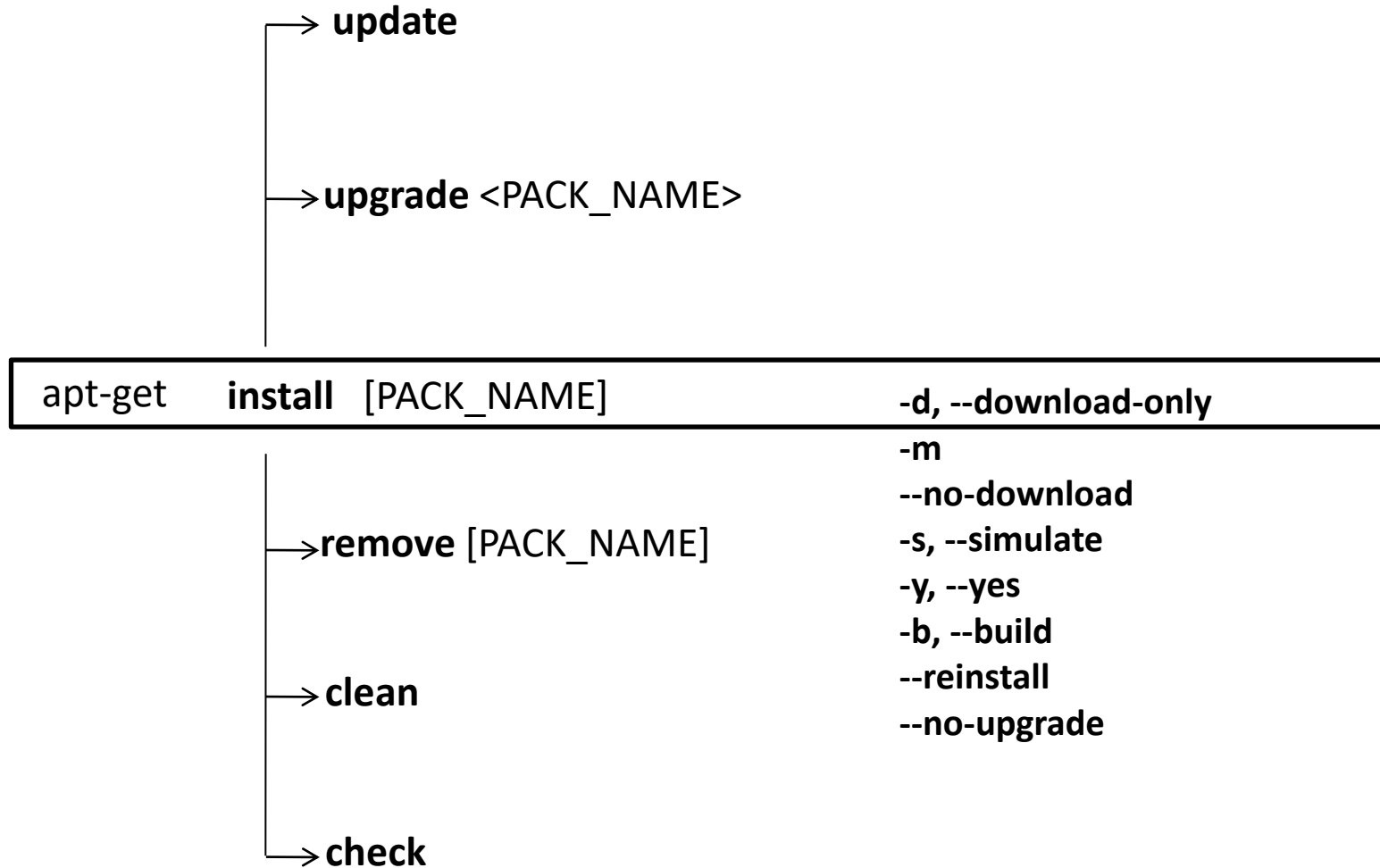
- устанавливать, удалять и обновлять пакеты
- разрешать зависимости
- искать пакеты по заданным критериям
- просматривать подробную информацию о пакете
- манипулировать ключами от репозиториев

ПАКЕТНЫЙ МЕНЕДЖЕР APT

- /etc/apt/apt.conf*** — основной файл настроек, используемый всеми инструментами из состава APT.
- /etc/apt/apt.conf.d*** — директория содержащая в себе файлы конфигурации, аналогичные по синтаксису **apt.conf**. При помощи этой директории можно быстро и удобно манипулировать настройками APT добавляя или удаляя подготовленные файлы с настройками.
- /etc/apt/auth.conf*** — файл, содержащий ключи, для авторизации в репозиториях. Например, туда добавляются логины и пароли от репозитория к купленным в Центре приложений программам
- /etc/apt/sources.list*** — файл с перечнем репозитория.
- /etc/apt/sources.list.d*** директория с файлами репозитория, по назначению аналогичных sources.list. Каждый репозиторий описывается в отдельном файле.
- /etc/apt/trusted.gpg.d*** — директория для хранения gpg-ключей репозитория
- /var/cache/apt/archives/*** — директория для хранения скачанных архивов
- /var/cache/apt/archives/partial/*** — директория для хранения архивов, находящихся в процессе скачивания

УТИЛИТА APT-GET

Описание: управление установкой пакетов из сетевых хранилищ с разрешением зависимостей



УТИЛИТА APT-GET

Форма:

`apt-get [options] [keys]`

Ключи:

- | | |
|--------------------------|--|
| update | – добавляет список пакетов из PATH_FILE в кэш |
| upgrade | – выводит краткую информацию по пакету PACK_NAME |
| install PACK_NAME | – краткая информация о неудовлетворенных зависимостях в кэше |
| remove PACK_NAME | – выводит информацию о пакетах, в названии которых содержится PACK_NAME |
| check | – список зависимостей пакета PACK_NAME |
| clean | – Выводит список имен пакетов, содержащих подстроку PACK_NAME. Если параметра нет, то выводит список всех пакетов в кэше |

Опции:

- | | |
|----------------------------|---|
| -d, --download-only | – только скачать пакеты и не устанавливать их |
| -m | – игнорировать отсутствующие зависимости |
| --no-download | – использовать только уже скачанные пакеты и не скачивать новые |
| -s, --simulate | – имитировать установку, но не производить ее |

УТИЛИТА APT-GET

- y, --yes** — добавляет список пакетов из PATH_FILE в кэш
- b, --build** — выводит краткую информацию по пакету PACK_NAME
- reinstall** — краткая информация о неудовлетворенных зависимостях в кэше
- s, --simulate** — выводит информацию о пакетах, в названии которых содержится PACK_NAME
- no-upgrade** — список зависимостей пакета PACK_NAME
- reinstall** — Выводит список имен пакетов, содержащих подстроку PACK_NAME. Если параметра нет, то выводит список всех пакетов в кэше

УТИЛИТА APT-CACHE

Описание:

Поиск и получение информации о пакетах, находящихся в репозиториях

Форма:

`apt-cache [options] [keys]`

Ключи:

- | | |
|-----------------------------|--|
| add PATH_FILE | – добавляет список пакетов из PATH_FILE в кэш |
| showpkg PACK_NAME | – выводит краткую информацию по пакету PACK_NAME |
| unmet | – краткая информация о неудовлетворенных зависимостях в кэше |
| search PACK_NAME | – выводит информацию о пакетах, в названии которых содержится PACK_NAME |
| depends PACK_NAME | – список зависимостей пакета PACK_NAME |
| pkgnames <PACK_NAME> | – Выводит список имен пакетов, содержащих подстроку PACK_NAME. Если параметра нет, то выводит список всех пакетов в кэше |

УТИЛИТА APT-KEY

Описание:

Служит для добавления ключей от репозиториев в систему. Ключи защищают репозитории от возможности подделки пакета.

Форма:

`apt-key [keys]`

Ключи:

- | | |
|------------------------------|---|
| <code>add PGP_KEY</code> | – добавление ключа в базу доверенных ключей |
| <code>del PGP_KEY_ID</code> | – удаление ключа из базы доверенных ключей репозиториев |
| <code>list, finger</code> | – вывести все pgp-ключи репозиториев |
| <code>adv PGP_KEY_URL</code> | – Получить ключ из источника (напр. сетевой URL) |

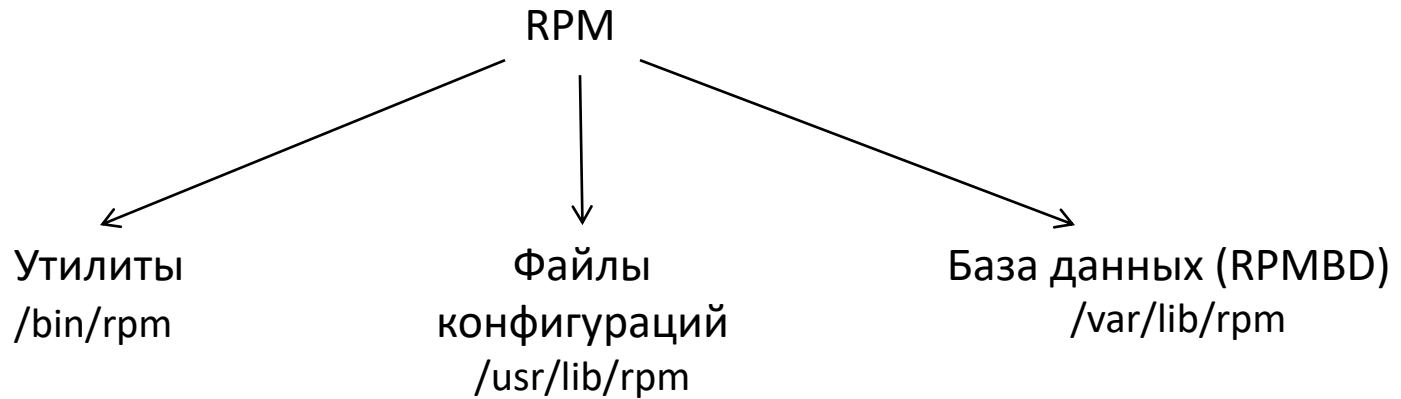
Пример:

```
#добавление ключа репозитория
apt-key add Repo-key.gpg

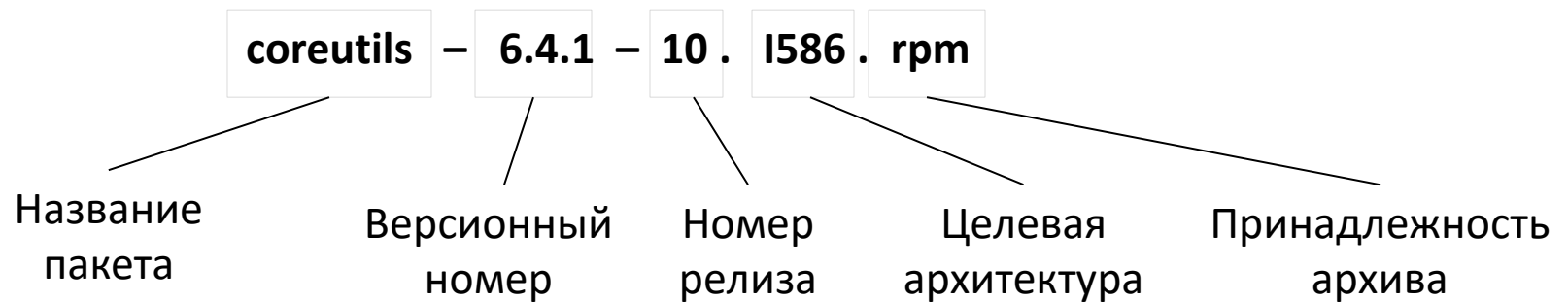
#просмотреть информацию о ключах репозиториев
apt-key fingerprint
```

МЕНЕДЖЕР RPM-ПАКЕТОВ

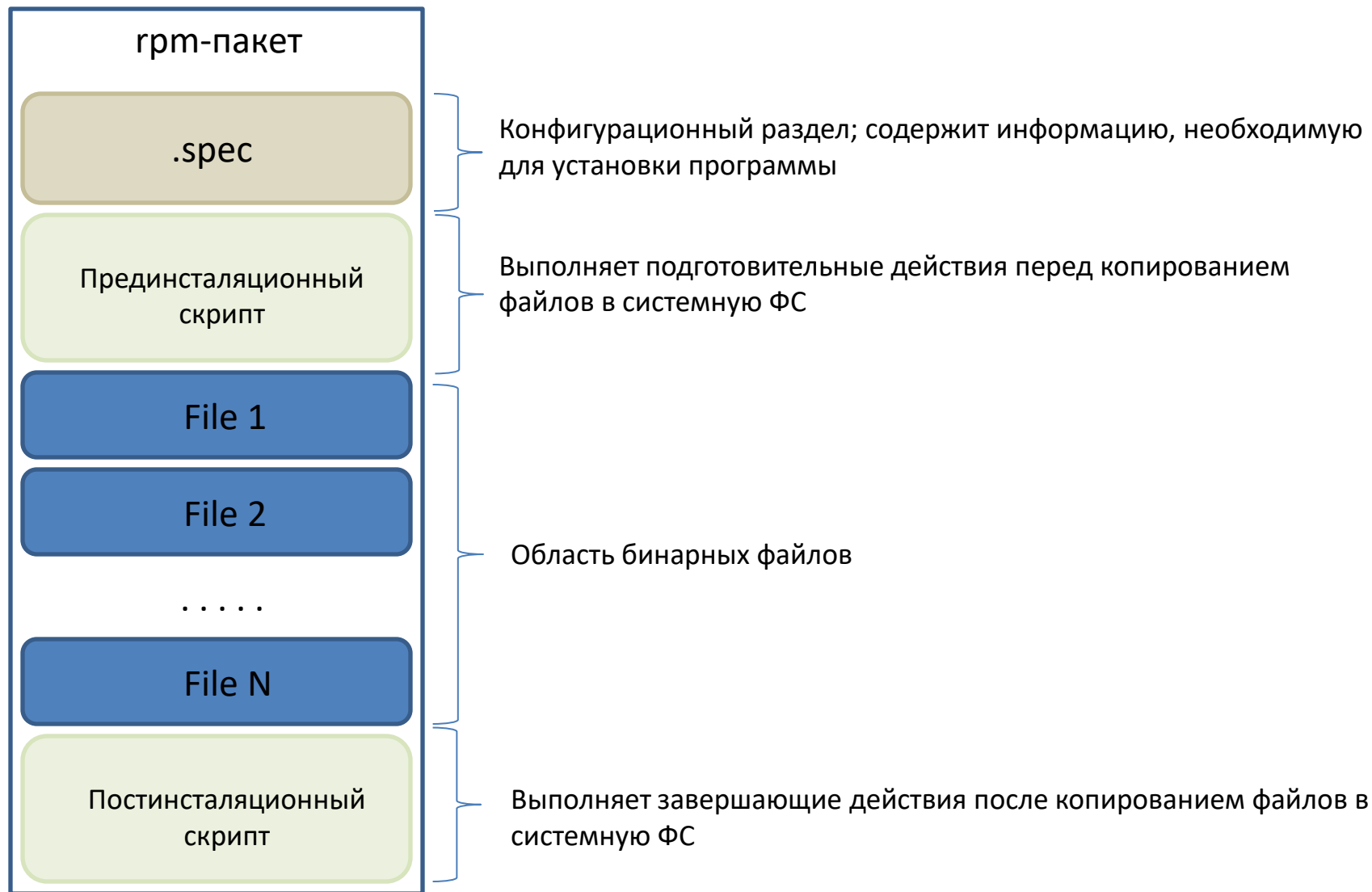
МЕНЕДЖЕР RPM-ПАКЕТОВ



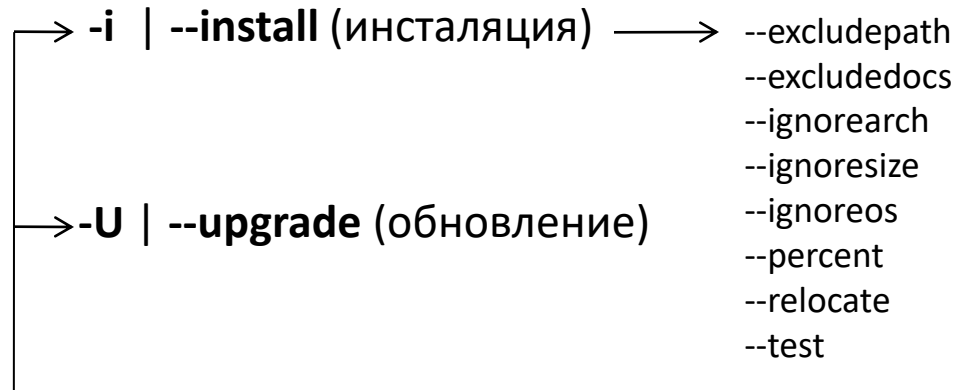
Структура имени пакета:



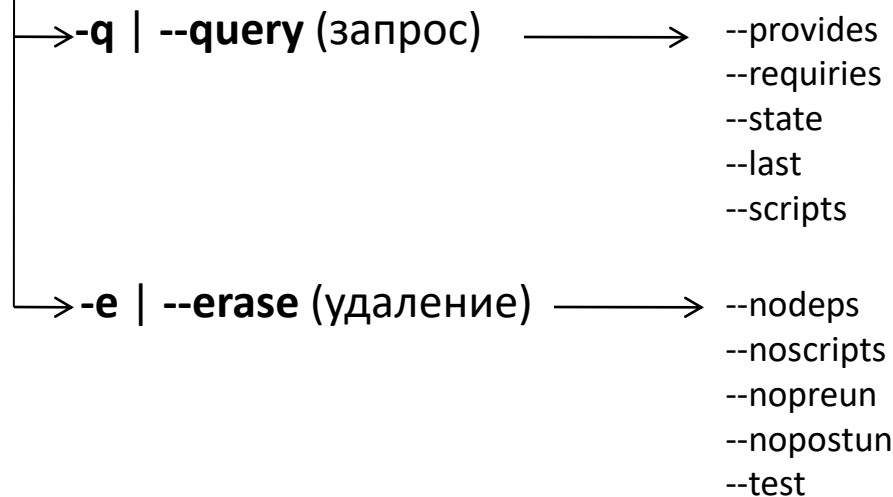
СТРУКТУРА RPM-ПАКЕТОВ



УПРАВЛЕНИЕ МЕНЕДЖЕРОМ ПАКЕТОВ



rpm	-V --verify (верификация)	--nodeps --noscripts	[имя_пакета]
-----	------------------------------------	--	---------------------



ИНСТАЛЯЦИЯ ПАКЕТОВ

Форма:

```
rpm -i [select options] [install options] package_file  
rpm -U [select options] [install options] package_file
```

Ключи:

- excludepath** OLD_PATH – не устанавливать файлы, имена которых начинаются с OLD_PATH
- excludedocs** – не устанавливать файлы документации
- ignoresize** – не проверять есть ли на диске свободное место для установки
- ignorearch** – не проверять возможность установки пакета на данную процессорную архитектуру
- ignoreos** – не проверять возможность установки на данную ОС
- includedocs** – установить только документацию
- justdb** – модифицировать только базу данных
- nodeps** – не проверять при установке зависимости
- noscripts** – не запускать скрипты
- oldpackage** – заменить пакет на более старую версию
- percent** – отобразить ход установки
- prefix** NEW_PATH
- relocate** OLD_PATH=NEW_PATH
- test** – пробная установка пакета

УДАЛЕНИЕ ПАКЕТОВ

Форма:

```
rpm -e | --erase [erase options] package_file
```

Ключи:

--nodeps – не проверять зависимости при удалении пакета

--noscripts – не выполнять скрипты при удалении; возможно отключение отдельных скриптов

--repackage – создать резервный rpm-пакет; используется директория по умолчанию

--test – пробное удаление пакета

Пример:

[illegible]

```
rpm -e --test --repackage openssh-4.4-1.i586.rpm #создание rpm-пакета openssh-4.4-1.i586.rpm
#используя данные RPMDB
```

ПОИСК ПАКЕТОВ И ПОЛУЧЕНИЕ ИНФОРМАЦИИ ИЗ RPMDB

Форма:

```
rpm -q | --query [select options] [query options] package_file
```

Ключи:

[SELECT OPTIONS]

-a – вывести все установленные пакеты

-f FILE_NAME – вывести список пакетов, которые используют файл FILE_NAME

-g GROUP - вывести список пакетов, сгруппированных под именем GROUP

-p PACKAGE – брать информацию не из RPMDB, а из самого rpm-пакета

--whatprovides CAPABILITY – вывести список пакетов, которые экспортируют зависимость CAPABILITY

--whatrequires CAPABILITY - вывести список пакетов, которые зависят от CAPABILITY

[QUERY OPTIONS]

-c – вывести только список конфигурационных файлов пакета

-d – вывести список только информационных файлов пакета, включая man-страницы

-l - вывести список файлов, которые будут установлены

-i – вывести подробную информацию о пакете

--last – вывести список установленных пакетов

--provides – зависимости, экспортируемые данным пакетом (package_file)

--requires – список пакетов, от которых зависит данный пакет (package_file)

--scripts – вывести содержимое скриптов, содержащихся в пакете

--state – вывести статус установленных файлов

ПРОВЕРКА ЦЕЛОСТНОСТИ ПАКЕТОВ

Форма:

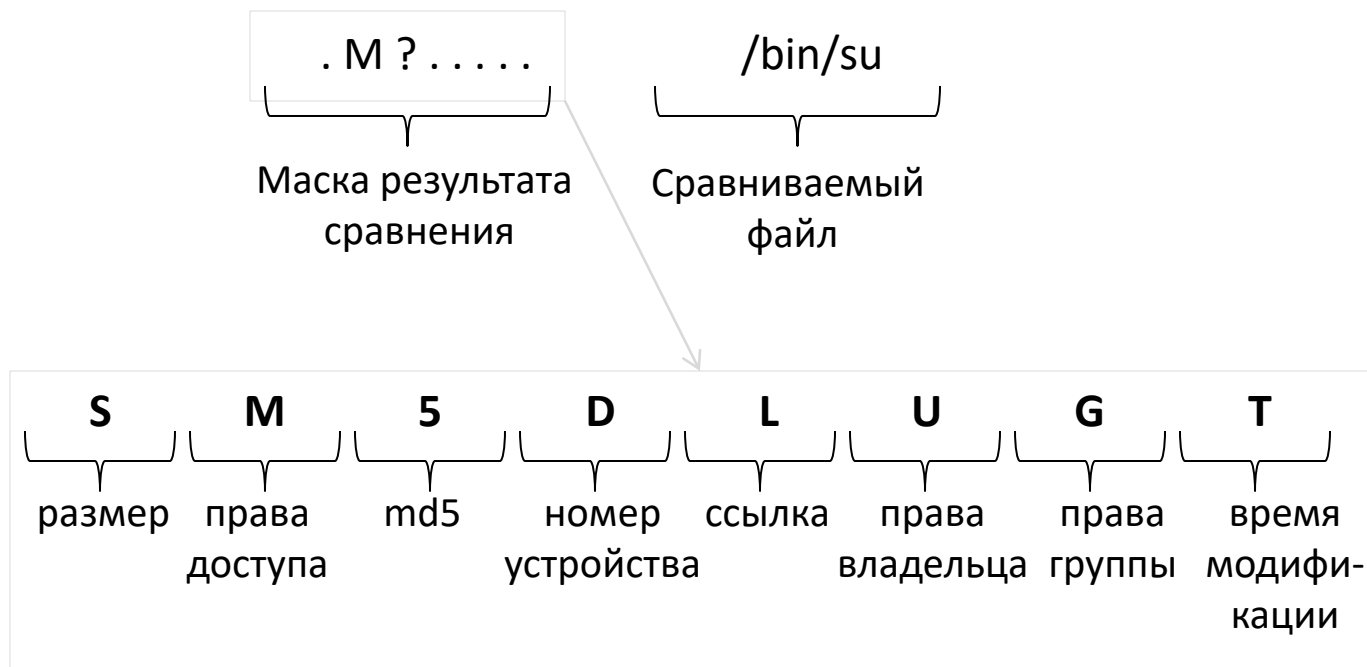
```
rpm -V | --verify [verify options] package_file
```

Ключи:

--nodeps — не проверять зависимости при верификации

--noscripts — отключить выполнение скрипта %verifyscript

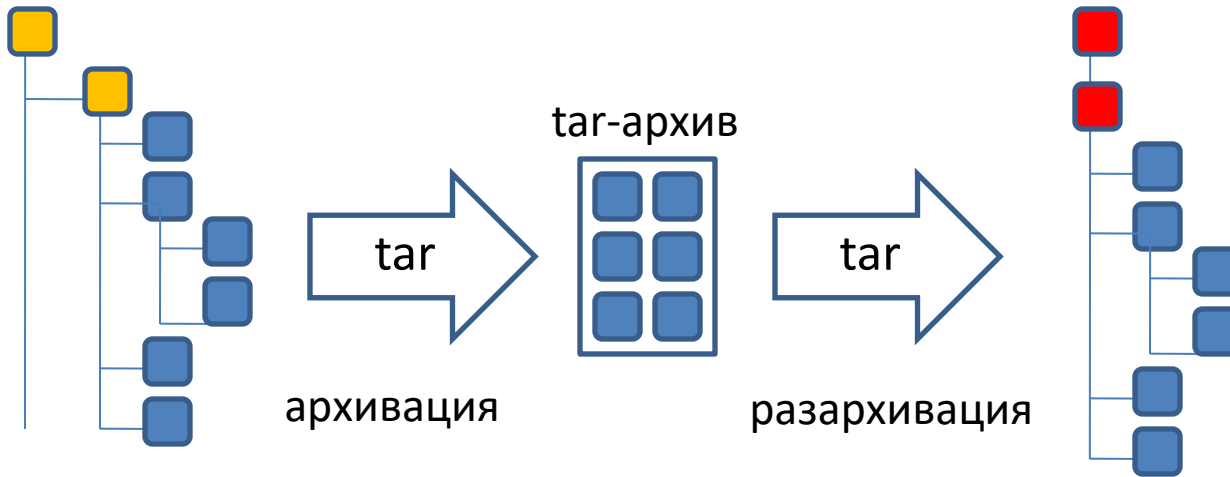
Формат вывода:



ПОСТАВКА АРХИВОМ

Архиватор tar

Используется для создания архива группы файлов с сохранением начальных прав доступа и файловой структуры



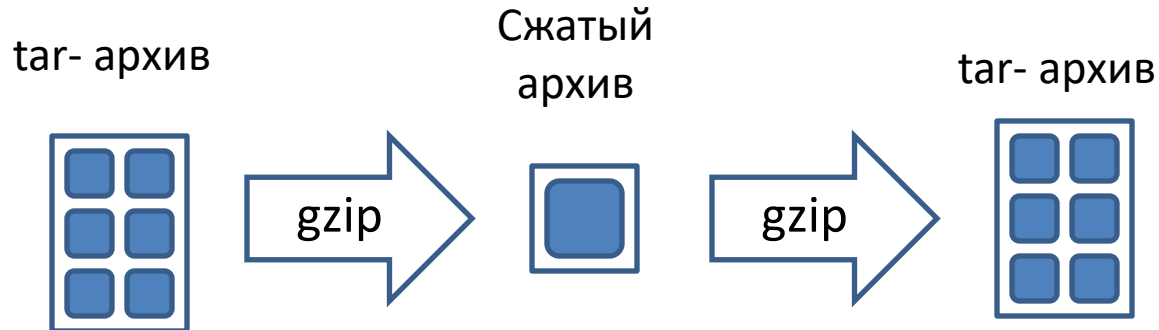
Форма: tar [Keys] [Files | Directories]

Ключи:

- c | **--create** – Создание нового архива
- u | **--update** – обновление содержимого архива
- x | **--extract** – извлечение содержимого архива
- t | **--list** – просмотр содержимого архива
- delete** – удаление файла или директории из архива
- append** – добавить файлы в конец архива
- C Dir – сменить текущую директорию на Dir
- f ArchName – задать имя используемого архива
- O – вывести содержимое архива в стандартный ввод

КОМПРЕССОР GZIP

Основное назначение - сжатие файлов



Форма: `gzip [Keys] [Files | Directories]`

Ключи:

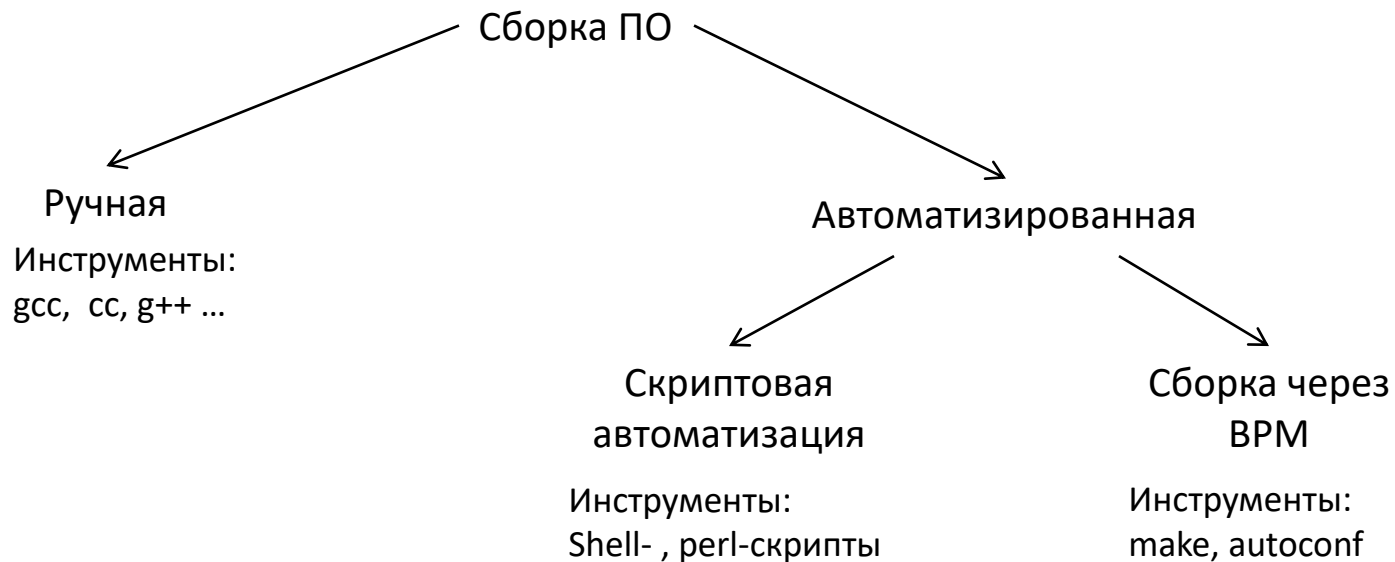
- d | --decompress** – Создание нового архива
- c | --stdout** – обновление содержимого архива
- r** – рекурсивная обработка директорий
- t | --list** – просмотр содержимого архива
- test** – проверка целостности архива
- l** – вывод информации о сжатом файле

Пример:

`gzip -c My.txt > My.txt.gz` #сжатие файла My.txt и запись содержимого в My.txt.gz

ПОСТАВКА ИСХОДНОГО КОДА

Исходный код поставляется конечным пользователям в виде тарболов, с указаниями сборки и установки программы



СТРУКТУРА MAKE-ФАЙЛА

- Представляет собой текстовый файл с инструкциями по управлению программным проектом в процессе сборки
- Разбит на целевые блоки; каждый блок описывает один этап сборки программы

Формат файла:

Цель_этапа : Список_компонентов
Набор_действий_для_данного_этапа

Пример:

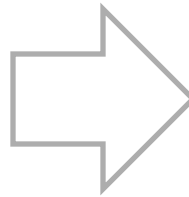
```
main.o : main.c stdio.h
gcc -c main.c stdio.h
```

Целевые блоки могут иметь сокращенный вид:

```
prog: main.o strprint.o
gcc -o prog main.o strprint.o

main.o : main.c stdio.h
gcc -c main.c strprint.h

strprint.o : strprint.c stdio.h
gcc -c strprint.c stdio.h
```



```
prog: main.o strprint.o
gcc -o prog main.o strprint.o

main.o : main.c stdio.h

strprint.o : strprint.c stdio.h
```

ПРИМЕР MAKE-ФАЙЛА

OBJS = main.o supp.o

BINS = prog

PREFIX = /usr/local

INSTALL = install

INSOPTS = -s -m 755 -o 0 -g 0 CC = gcc

.PHONY = all clean install

all: \$(BINS)

prog: \$(OBJS)

\$(CC) -o prog \$(OBJS)

main.o: defs.h

supp.o: defs.h supp.h

clean:

rm -f \$(BINS)

rm -f \$(OBJS)

rm -f *~

install:

for \$i in \$(BINS) ; do \

\$(INSTALL) \$(INSOPTS) \$\$i \$(PREFIX) /bin ; \

done

КОМАНДА MAKE

Форма:

`make [ -f makefile ] [ Keys ] Target`

Ключи:

- C DIR – изменение текущей директории проекта
- d – печать сборочной информации
- i – игнорировать ошибки в командах
- k – при возникновении ошибки продолжить сборку насколько это возможно
- o FILE – не пересобирать файл, даже если он старше, чем все остальные файлы проекта
- p – вывести на экран таблицу порядка сборки
- t – изменить время модификации файлов проекта

Пример:

`make -df /root/Desktop/Program/Makefile -o main.o, strio.o`